PROJECT REPORT

PERSONAL EXPENSE TRACKER

(with Shared Expense Detection)

Submitted By

Name: Monu Kumar
UID: 25BCD10170
Programme: BCA (Data Science)
Semester: 3rd Semester
Section: 25BCD-3(B)

Submitted To

Faculty Name: Abhinay Sood

University Institute of Computing

Chandigarh University

CERTIFICATE / DECLARATION

This is to certify that the mini project entitled **“Personal Expense Tracker with Shared Expense Fairness Detection”** has been prepared by Monu Kumar, UID 25BCD10170, a student of BCA (Data Science), 3rd Semester, Chandigarh University, as part of the Object-Oriented Programming Using C++ coursework.

The project demonstrates the application of object-oriented programming concepts to a real-world expense management problem. It includes personal expense tracking, shared expense management, balance calculation, and a Shared Expense Fairness Detector.

The project has been developed for academic purposes and demonstrates the use of classes, objects, constructors, encapsulation, collections, functions, and file handling.

Student Name: Monu Kumar
UID: 25BCD10170

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my OOPS (C++) faculty for providing me with the opportunity to develop this mini project.

This project helped me understand how programming concepts can be used to solve everyday problems. While developing the Personal Expense Tracker, I learned about classes, objects, constructors, encapsulation, vectors, maps, and file handling in C++.

I am also thankful to Chandigarh University for providing an academic environment that encourages practical learning and problem-solving.

Finally, I acknowledge the C++ documentation and learning resources that helped me understand the language and its standard libraries.

ABSTRACT

The **Personal Expense Tracker with Shared Expense Fairness Detection** is a console-based application developed using C++. Its main purpose is to help users record personal expenses, calculate their total spending, manage shared group expenses, and determine the amount each group member should pay or receive.

Managing expenses can become confusing when multiple people share costs for food, travel, accommodation, or other activities. People may find it difficult to remember who paid, how much each person owes, and whether the expenses have been divided fairly.

This project addresses the problem through a simple menu-driven application. It provides seven main features: adding personal expenses, viewing personal expenses, calculating personal spending totals, adding shared expenses, viewing shared expenses, checking member balances, and generating a Shared Expense Fairness Report.

The main feature of the project is the Shared Expense Fairness Detector. It calculates each member's net balance by dividing shared expenses equally among the participating members and considering the amount paid by each person.

The project demonstrates important C++ concepts, including classes, objects, constructors, encapsulation, vectors, maps, loops, conditional statements, and file handling. It provides a foundation for developing more advanced expense management applications in the future.

TABLE OF CONTENTS

S. No.	Chapter / Section
1	Introduction
2	Problem Statement
3	Objectives
4	System Design
5	UML Class Diagram
6	OOP Concepts Used
7	Algorithm and Flowchart
8	Implementation Details
9	Unique Feature: Shared Expense Fairness Detector
10	Source Code
11	Sample Inputs and Outputs
12	Testing and Results
13	Limitations
14	Future Scope
15	Conclusion
16	References

1. INTRODUCTION

Managing personal finances is an important part of everyday life. People regularly spend money on food, transportation, education, shopping, entertainment, and other necessities. Without proper records, it can become difficult to understand how much money has been spent.

The problem becomes more complicated when expenses are shared among friends, roommates, classmates, or family members. For example, if one person pays for dinner for a group, the amount must be divided among the people who participated. When several people pay for different expenses, manually calculating everyone's final balance can become confusing.

The Personal Expense Tracker with Shared Expense Fairness Detection is designed to address these problems through a simple C++ application.

The application allows users to record personal expenses, view their recorded expenses, calculate their total spending, and manage shared group expenses. It also calculates the net balance of each group member and identifies who should pay or receive money.

The project has been developed using Object-Oriented Programming principles. Its menu-driven interface makes the application easy to operate and suitable for demonstrating basic C++ programming concepts.

1.1 Purpose of the Project

The purpose of this project is to develop a basic expense management system that simplifies personal spending records and shared expense calculations.

It also aims to demonstrate how OOP concepts can be applied to a practical problem rather than using only separate functions and variables.

1.2 Scope of the Project

The project covers the following activities:

- Recording personal expenses.
- Displaying personal expense records.
- Calculating personal spending totals.
- Recording shared group expenses.
- Displaying shared expense records.

- Calculating the net balance of group members.
- Generating a fairness report showing amounts to pay or receive.

The current version is a console-based application intended for academic demonstration. It saves newly entered records in text files but does not automatically load previously saved records when restarted.

2. PROBLEM STATEMENT

People often record expenses informally or depend on memory to track their spending. This can make it difficult to determine total expenditure and understand spending patterns.

Shared expenses create an additional challenge. When multiple people participate in a group activity, one member may pay the entire amount while the others are expected to contribute their respective shares.

For example, three friends may spend ₹900 on dinner, with one friend paying the entire bill. Each person is responsible for ₹300 if the expense is shared equally. However, the person who paid should receive contributions from the other two members.

When several shared expenses are involved, calculating the final amount owed by each person becomes more complicated.

The problem addressed by this project is to develop a C++ application that records personal and shared expenses, calculates spending totals, divides shared expenses equally, and displays the net amount each participant should pay or receive.

2.1 Proposed Solution

The proposed system provides a menu-driven interface through which users can perform personal and shared expense operations.

The Shared Expense Fairness Detector calculates the net balance of each group member across the shared expenses recorded during the current session. It helps users understand their financial position without manually calculating every expense.

3. OBJECTIVES

The main objectives of the project are:

1. To develop a simple expense tracking application using C++.

2. To allow users to record personal expenses with categories, descriptions, and amounts.
3. To display personal expenses entered during the current session.
4. To calculate total personal spending.
5. To record shared expenses along with the payer and participating members.
6. To display shared expense records.
7. To calculate the net balance of each group member.
8. To identify which members should pay money and which should receive money.
9. To demonstrate Object-Oriented Programming concepts in a practical application.
10. To use basic file handling to save personal and shared expense records.

4. SYSTEM DESIGN

The application follows a menu-driven design. The user selects an operation from the main menu, and the program executes the corresponding function.

The system is divided into classes so that each class has a specific responsibility.

4.1 Main Components

A. Expense Class

The Expense class represents a personal expense.

It stores:

- Category
- Description
- Amount

It also provides a method to display the expense details.

B. GroupExpense Class

The GroupExpense class represents a shared expense.

It stores:

- Name of the payer
- Expense description
- Total amount
- Names of participating members

It provides a method to display the shared expense information.

C. FileManager Class

The FileManager class handles saving expense records to text files.

It saves personal expenses in personal_expenses.txt and shared expenses in group_expenses.txt.

D. ExpenseTracker Class

The ExpenseTracker class manages personal expenses.

Its main operations are:

- Add personal expense.
- View personal expenses.
- Calculate personal spending summary.

E. SharedExpenseManager Class

The SharedExpenseManager class manages group expenses.

Its main operations are:

- Add shared expense.
- View shared expenses.
- Calculate member balances.
- Display member balances.
- Generate a fairness report.

F. Main Function

The main() function displays the menu and accepts the user's choice. It creates objects of ExpenseTracker and SharedExpenseManager and calls their methods according to the selected option.

4.2 Working of the System

The program begins by displaying the main menu.

If the user selects a personal expense operation, the ExpenseTracker class performs the requested task.

If the user selects a shared expense operation, the SharedExpenseManager class handles the input, records the expense, or calculates the member balances.

The program continues displaying the menu until the user selects the exit option.

5. UML CLASS DIAGRAM

A UML class diagram represents the structure of the application by showing its classes, attributes, methods, and relationships.

For this project, the diagram should contain five classes:

1. Expense
2. GroupExpense
3. FileManager
4. ExpenseTracker
5. SharedExpenseManager

5.1 Class Structure

Class 1: Expense

Attributes:

- category : string
- description : string
- amount : double

Methods:

- Expense(category, description, amount)
- display() : void

Class 2: GroupExpense

Attributes:

- payer : string

- description : string
- amount : double
- members : vector<string>

Methods:

- GroupExpense(payer, description, amount, members)
- display() : void

Class 3: FileManager

Methods:

- savePersonalExpense(Expense) : void
- saveGroupExpense(GroupExpense) : void

Both saving methods are static in the implementation.

Class 4: ExpenseTracker

Private Attribute:

- expenses : vector<Expense>

Public Methods:

- addExpense() : void
- viewExpenses() : void
- showSummary() : void

Class 5: SharedExpenseManager

Private Attribute:

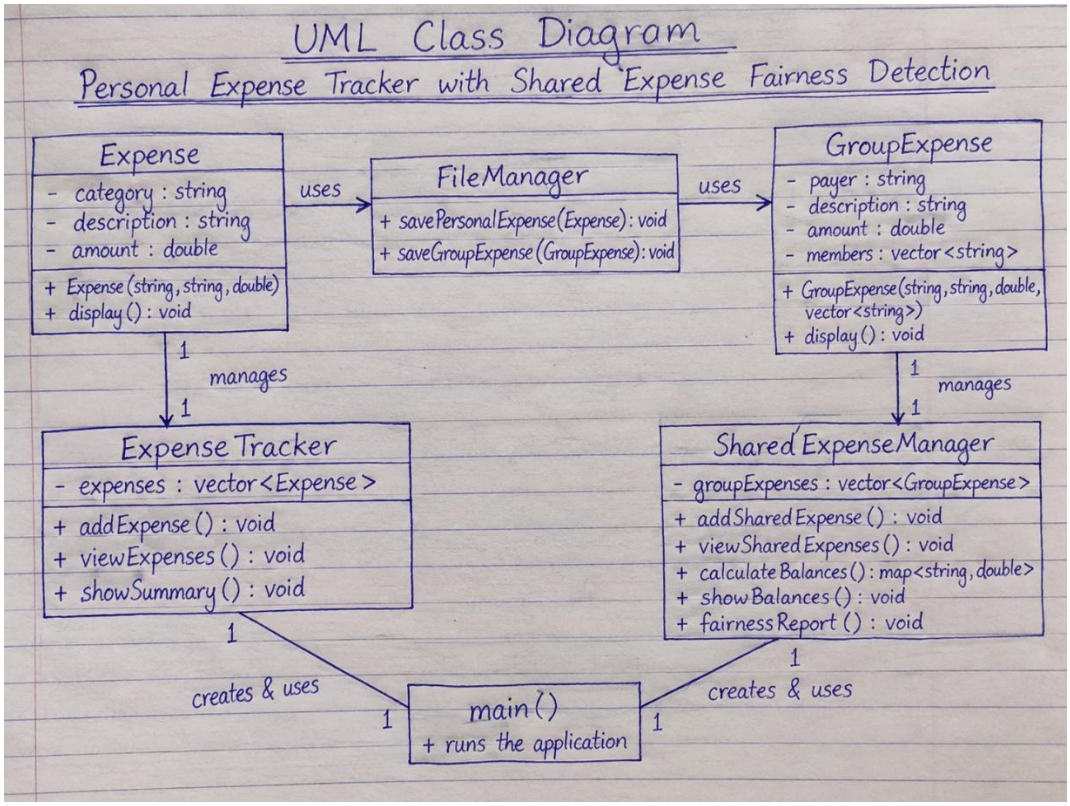
- groupExpenses : vector<GroupExpense>

Public Methods:

- addSharedExpense() : void
- viewSharedExpenses() : void
- calculateBalances() : map<string, double>
- showBalances() : void
- fairnessReport() : void

5.2 Relationships Between Classes

- ExpenseTracker contains a collection of Expense objects.
- SharedExpenseManager contains a collection of GroupExpense objects.
- ExpenseTracker uses FileManager to save personal expenses.
- SharedExpenseManager uses FileManager to save shared expenses.
- The main() function creates and uses objects of both manager classes.



6. OOP CONCEPTS USED

The project demonstrates several important Object-Oriented Programming concepts.

6.1 Classes and Objects

A class is a blueprint that defines the attributes and behaviour of an object.

The project uses classes such as Expense, GroupExpense, ExpenseTracker, and SharedExpenseManager.

Objects are created from these classes to store expense information and perform operations.

For example, an object of the Expense class stores the category, description, and amount of a personal expense.

6.2 Encapsulation

Encapsulation combines data and related functions within a class.

In this project, the expense collections inside ExpenseTracker and SharedExpenseManager are private. They are accessed through public methods.

This helps organise the program and prevents the collections from being directly modified from outside the class.

6.3 Abstraction

Abstraction hides implementation details and provides simple operations to the user.

For example, the user selects the fairness report option without needing to understand the internal balance calculation.

The program performs the calculation through the fairnessReport() and calculateBalances() methods.

6.4 Constructors

Constructors initialise objects when they are created.

The Expense constructor assigns values to the category, description, and amount attributes.

Similarly, the GroupExpense constructor initialises the payer, description, amount, and member list.

6.5 Vectors

A vector is a dynamic collection provided by the C++ Standard Template Library.

The project uses vectors to store personal expense objects, shared expense objects, and group member names.

Vectors allow the program to manage a variable number of records.

6.6 Maps

The map container stores key-value pairs.

In the fairness calculation, the member's name is used as the key and their net balance is stored as the value.

This allows the program to calculate and display the balance of each member.

6.7 File Handling

The project uses file handling to save records.

The ofstream class is used to write data into text files. Append mode allows new records to be added without intentionally overwriting the existing file contents.

The current implementation does not read saved records back into the program when it starts.

7. ALGORITHM AND FLOWCHART

7.1 Main Program Algorithm

Step 1: Start the program.

Step 2: Create an Expense Tracker object and a Shared Expense Manager object.

Step 3: Display the main menu.

Step 4: Accept the user's choice.

Step 5: If the choice is 1, add a personal expense.

Step 6: If the choice is 2, display personal expenses.

Step 7: If the choice is 3, calculate and display the personal spending total.

Step 8: If the choice is 4, add a shared expense.

Step 9: If the choice is 5, display shared expenses.

Step 10: If the choice is 6, calculate and display member balances.

Step 11: If the choice is 7, generate the Shared Expense Fairness Report.

Step 12: If the choice is 0, display the exit message and stop.

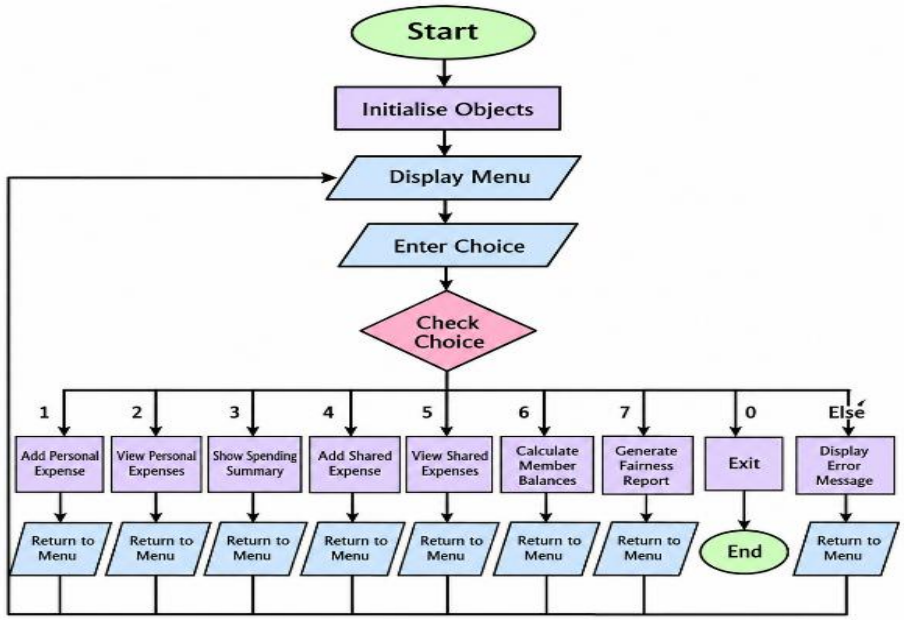
Step 13: If another value is entered, display an invalid-choice message.

Step 14: Repeat the menu until the user selects 0.

7.2 Fairness Calculation Algorithm

1. Create an empty map of member balances.
2. Process each shared expense.
3. Calculate the equal share by dividing the expense amount by the number of participating members.
4. Subtract the equal share from each participating member's balance.
5. Add the total amount paid to the payer's balance.
6. Repeat the process for every shared expense.
7. Display the final balance of each member.
8. Interpret a positive balance as money to receive.
9. Interpret a negative balance as money to pay.
10. Display the fairness report.

7.3 Flowchart



8. IMPLEMENTATION DETAILS

The project is implemented in C++ using a single source file named main.cpp.

The program uses a menu-driven structure, a switch statement, loops, classes, vectors, maps, and file handling.

8.1 Add Personal Expense

The user enters the category, description, and amount.

The program validates that the amount is a valid positive number. If the amount is valid, an Expense object is created.

The object is added to the personal expense vector and saved to personal_expenses.txt.

8.2 View Personal Expenses

The program displays personal expenses recorded during the current session.

Each record includes the category, description, and amount.

If no personal expenses have been added during the current session, the program displays an appropriate message.

8.3 Personal Spending Summary

The program calculates the total by adding the amounts of all personal expense objects stored in the current session.

The calculated total is displayed in rupees.

8.4 Add Shared Expense

The user enters:

- Expense description.
- Name of the person who paid.
- Total amount.
- Number of participating members.
- Names of the members.

The program checks that the amount is positive, the number of members is valid, and the payer is included in the group.

If the information is valid, a GroupExpense object is created and saved to group_expenses.txt.

8.5 View Shared Expenses

The program displays each shared expense recorded during the current session, including the payer, description, total amount, and participating members.

8.6 View Member Balances

The program calculates the net balance for every member.

A positive balance indicates that the member should receive money. A negative balance indicates that the member should pay money.

A balance close to zero indicates that the member has no outstanding amount under the current calculation.

8.7 Shared Expense Fairness Report

The report summarises the balance of every member.

It identifies the amount each person should pay or receive and indicates whether the group has outstanding amounts.

The report is based on equal sharing among the members listed for each expense.

8.8 Data Files

The program uses the following files:

File Name	Purpose
main.cpp	Contains the complete C++ source code
personal_expenses.txt	Stores personal expense records
group_expenses.txt	Stores shared expense records

The text files are created or updated when valid expenses are added. The current version does not automatically reload these records into memory after restarting the program.

9. UNIQUE FEATURE: SHARED EXPENSE FAIRNESS DETECTOR

The **Shared Expense Fairness Detector** is the main distinguishing feature of this project.

Many basic expense trackers only record spending or calculate the total amount paid. This project goes a step further by calculating the net balance of each group member across multiple shared expenses.

The feature helps answer three questions:

1. Who paid for the shared expenses?
2. How much should each member contribute?
3. Which members should pay money or receive money?

9.1 Working Principle

For each shared expense, the program calculates the equal share:

Equal Share = Total Expense ÷ Number of Participating Members

It then subtracts the equal share from every participating member's balance.

The payer is credited with the full amount paid.

After all expenses have been processed, the resulting balances show each member's net financial position.

- Positive balance: The member should receive money.
- Negative balance: The member should pay money.
- Zero balance: The member has no outstanding amount.

9.2 Example

Suppose three friends, Aman, Riya, and Karan, share a dinner expense of ₹900.

Aman pays the entire amount.

Total Expense: ₹900
Number of Members: 3
Equal Share: ₹900 ÷ 3 = ₹300

The calculated balances are:

Member	Amount Paid	Equal Share	Net Balance
Aman	₹900	₹300	+₹600
Riya	₹0	₹300	−₹300
Karan	₹0	₹300	−₹300

The fairness report shows:

- Aman should receive ₹600.
- Riya should pay ₹300.
- Karan should pay ₹300.

The total amount to be paid equals the total amount to be received, so the balances are consistent with the equal-split calculation.

9.3 Benefits

The Shared Expense Fairness Detector:

- Reduces manual calculation.
- Makes shared expenses easier to understand.
- Shows the net balance of each member.
- Helps reduce confusion about contributions.
- Supports multiple shared expenses in one session.
- Demonstrates practical problem-solving using C++.

9.4 Assumptions

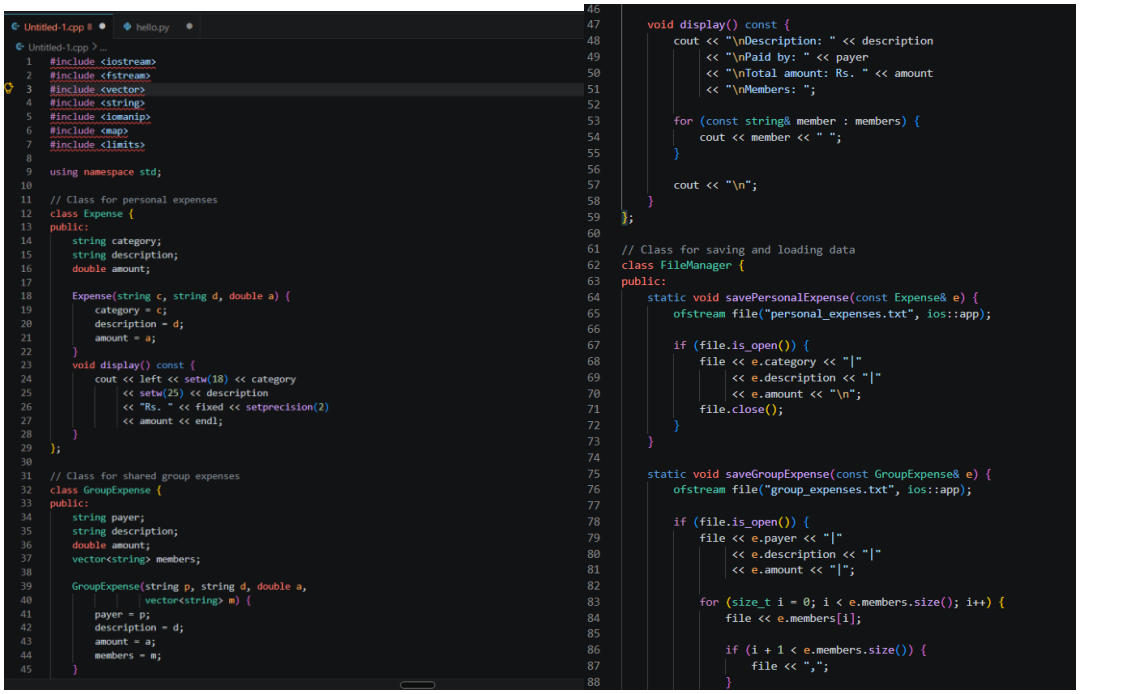
The current implementation assumes that:

- Every listed member shares an expense equally.

- The payer is included in the participating member list.
- Member names are entered consistently.
- Only expenses recorded during the current session are included in the report.

The system does not process actual payments or automatically mark outstanding balances as settled.

10.SOURCE CODE



```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <iomanip>
#include <map>
#include <limits>

using namespace std;

// Class for personal expenses
class Expense {
public:
    string category;
    string description;
    double amount;

    Expense(string c, string d, double a) {
        category = c;
        description = d;
        amount = a;
    }
    void display() const {
        cout << left << setw(18) << category
             << setw(25) << description
             << "Rs. " << fixed << setprecision(2)
             << amount << endl;
    }
};

// Class for shared group expenses
class GroupExpense {
public:
    string payer;
    string description;
    double amount;
    vector<string> members;

    GroupExpense(string p, string d, double a,
                 vector<string> m) {
        payer = p;
        description = d;
        amount = a;
        members = m;
    }
    void display() const {
        cout << "\nDescription: " << description
             << "\nPaid by: " << payer
             << "\nTotal amount: Rs. " << amount
             << "\nMembers: ";

        for (const string& member : members) {
            cout << member << " ";
        }

        cout << "\n";
    }
};

// Class for saving and loading data
class FileManager {
public:
    static void savePersonalExpense(const Expense& e) {
        ofstream file("personal_expenses.txt", ios::app);

        if (file.is_open()) {
            file << e.category << "|"
                 << e.description << "|"
                 << e.amount << "\n";
            file.close();
        }
    }

    static void saveGroupExpense(const GroupExpense& e) {
        ofstream file("group_expenses.txt", ios::app);

        if (file.is_open()) {
            file << e.payer << "|"
                 << e.description << "|"
                 << e.amount << "|";

            for (size_t i = 0; i < e.members.size(); i++) {
                file << e.members[i];

                if (i + 1 < e.members.size()) {
                    file << ",";
                }
            }

            file << "\n";
            file.close();
        }
    }
};

// Class for personal expense management
class ExpenseTracker {
private:
    vector<Expense> expenses;

public:
    void addExpense() {
        string category, description;
        double amount;

        cout << "Enter category (Food, Travel, etc.): ";
        getline(cin, category);

        cout << "Enter description: ";
        getline(cin, description);

        cout << "Enter amount: Rs. ";
        cin >> amount;

        if (cin.fail() || amount <= 0) {
            cout << "Invalid amount!\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return;
        }

        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        Expense e(category, description, amount);
        expenses.push_back(e);
        FileManager::savePersonalExpense(e);
        cout << "Personal expense added successfully!\n";
    }

    void viewExpenses() const {
        if (expenses.empty()) {
            cout << "No personal expenses added in this session.\n";
            return;
        }

        cout << "\n--- Personal Expenses ---\n";
        cout << left << setw(18) << "Category"
             << setw(25) << "Description"
             << "Amount\n";

        for (const Expense& e : expenses) {
            e.display();
        }
    }

    void showSummary() const {
        double total = 0;

        for (const Expense& e : expenses) {
            total += e.amount;
        }

        cout << "\nTotal personal spending in this session: Rs. "
             << fixed << setprecision(2) << total << "\n";
    }
};

// Class for shared expense management and fairness detection
class SharedExpenseManager {
private:
    vector<GroupExpense> groupExpenses;

public:
    void addSharedExpense() {
        string payer, description;
        double amount;
        int numberOfMembers;
```

(const char [47])"\nTotal personal spending in this session

Generate Copilot summary

```
        cout << "Enter expense description: ";
        getline(cin, description);

        cout << "Enter name of person who paid: ";
        getline(cin, payer);

        cout << "Enter total expense amount: Rs. ";
        cin >> amount;

        if (cin.fail() || amount <= 0) {
            cout << "Invalid amount!\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return;
        }

        cout << "Enter number of group members, including the payer: ";
        cin >> numberOfMembers;

        if (cin.fail() || numberOfMembers <= 0 ||
            numberOfMembers > 100) {
            cout << "Invalid number of members!\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return;
        }

        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        vector<string> members;
        bool payerFound = false;

        for (int i = 0; i < numberOfMembers; i++) {
            string name;
            cout << "Enter name of member " << i + 1 << ": ";
            getline(cin, name);

            members.push_back(name);

            if (name == payer) {
                payerFound = true;
            }
        }

        if (!payerFound) {
            cout << "Error: The payer must be included in the group.\n";
            return;
        }

        GroupExpense e(payer, description, amount, members);
        groupExpenses.push_back(e);
        FileManager::saveGroupExpense(e);

        cout << "Shared expense added successfully!\n";
    }

    void viewSharedExpenses() const {
        if (groupExpenses.empty()) {
            cout << "No shared expenses added in this session.\n";
            return;
        }

        cout << "\n--- Shared Expenses ---\n";

        for (const GroupExpense& e : groupExpenses) {
            e.display();
        }
    }

    map<string, double> calculateBalances() const {
        map<string, double> balances;

        for (const GroupExpense& e : groupExpenses) {
            if (e.members.empty()) {
                continue;
            }

            double share = e.amount / e.members.size();

            // Every member owes their equal share.
            for (const string& member : e.members) {
                balances[member] -= share;
            }

            // The payer gets credit for the amount paid.
            balances[e.payer] += e.amount;
        }

        return balances;
    }

    void showBalances() const {
        map<string, double> balances = calculateBalances();

        if (balances.empty()) {
            cout << "No shared expense data available.\n";
            return;
        }

        cout << "\n--- Member Balances ---\n";
        cout << "Positive balance = money to receive\n";
        cout << "Negative balance = money to pay\n\n";

        for (const auto& entry : balances) {
            cout << left << setw(20) << entry.first
                 << "Rs. " << fixed << setprecision(2)
                 << entry.second << "\n";
        }
    }

    void fairnessReport() const {
        map<string, double> balances = calculateBalances();

        if (balances.empty()) {
            cout << "No shared expense data available.\n";
            return;
        }

        cout << "\n--- Shared Expense Fairness Report ---\n";

        bool balanced = true;

        for (const auto& entry : balances) {
            if (entry.second > 0.01) {
                cout << "Should receive Rs. "
                     << fixed << setprecision(2)
                     << entry.second << "\n";
                balanced = false;
            }
            else if (entry.second < -0.01) {
                cout << "Should pay Rs. "
                     << fixed << setprecision(2)
                     << -entry.second << "\n";
                balanced = false;
            }
            else {
                cout << "Settled fairly\n";
            }
        }

        if (balanced) {
            cout << "\nAll members are financially balanced.\n";
        } else {
            cout << "\nThe group has pending amounts to settle.\n";
            cout << "The balances above show how much each member "
                 << "should pay or receive.\n";
        }
    }
};

int main() {
    ExpenseTracker personalTracker;
    SharedExpenseManager sharedManager;

    int choice;

    do {
        cout << "\n====================================\n";
        cout << "     PERSONAL EXPENSE TRACKER\n";
        cout << "====================================\n";
        cout << "1. Add Personal Expense\n";
```

```
        cout << "1. Add Personal Expense\n";
        cout << "2. View Personal Expenses\n";
        cout << "3. Personal Spending Summary\n";
        cout << "4. Add Shared Expense\n";
        cout << "5. View Shared Expenses\n";
        cout << "6. View Member Balances\n";
        cout << "7. Shared Expense Fairness Report\n";
        cout << "0. Exit\n";
        cout << "====================================\n";
        cout << "Enter your choice: ";

        cin >> choice;

        if (cin.fail()) {
            cout << "Please enter a valid number.\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }

        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        switch (choice) {
            case 1:
                personalTracker.addExpense();
                break;

            case 2:
                personalTracker.viewExpenses();
                break;

            case 3:
                personalTracker.showSummary();
                break;

            case 4:
                sharedManager.addSharedExpense();
                break;

            case 5:
                sharedManager.viewSharedExpenses();
                break;
```

```
            case 6:
                sharedManager.showBalances();
                break;

            case 7:
                sharedManager.fairnessReport();
                break;

            case 0:
                cout << "Thank you for using Personal Expense Tracker!\n";
                break;

            default:
                cout << "Invalid choice. Please try again.\n";
        }

    } while (choice != 0);

    return 0;
}
```

11. SAMPLE INPUTS AND OUTPUTS

The following examples illustrate the expected behaviour of the program. Add actual screenshots from your own program execution to this section before final submission.

11.1 Sample Output: Main Menu

```
==================================
     PERSONAL EXPENSE TRACKER
==================================
1. Add Personal Expense
2. View Personal Expenses
3. Personal Spending Summary
4. Add Shared Expense
5. View Shared Expenses
6. View Member Balances
7. Shared Expense Fairness Report
0. Exit
==================================
Enter your choice:
```

11.2 Sample Input: Personal Expense

```
Enter your choice: 1
Enter category (Food, Travel, etc.): food
Enter description: lunch
Enter amount: Rs. 150
Personal expense added successfully!
```

11.3 Sample Input: Shared Expense

```
Enter your choice: 4
Enter expense description: Dinner
Enter name of person who paid: Aman
Enter total expense amount: Rs. 900
Enter number of group members, including the payer: 3
Enter name of member 1: Aman
Enter name of member 2: Riya
Enter name of member 3: Karan
Shared expense added successfully!
```

11.4 Sample Output: Member Balances

```
Enter your choice: 6

--- Member Balances ---
Positive balance = money to receive
Negative balance = money to pay

Aman                Rs. 600.00
Karan               Rs. -300.00
Riya                Rs. -300.00
```

11.5 Sample Output: Fairness Report

```
--- Shared Expense Fairness Report ---
Aman: Should receive Rs. 600.00
Karan: Should pay Rs. 300.00
Riya: Should pay Rs. 300.00

The group has pending amounts to settle.
The balances above show how much each member should pay or receive.
```

11.6 Invalid Amount

```
Enter your choice: 1
Enter category (Food, Travel, etc.): food
Enter description: lunch
Enter amount: Rs. -100
Invalid amount!
```

12. TESTING AND RESULTS

Testing is performed to verify that the program accepts valid input, rejects certain invalid values, and calculates member balances correctly.

The following test cases should be executed and verified during the demonstration.

Test Case	Input / Action	Expected Result
1	Add a valid personal expense	Expense is added and saved

Test Case	Input / Action	Expected Result
2	Enter zero or a negative amount	Invalid amount message appears
3	View personal expenses after adding one	Current-session expense appears
4	Calculate personal spending summary	Correct total is displayed
5	Add a valid shared expense	Shared expense is accepted
6	Enter a payer who is not in the member list	Validation error appears
7	View shared expenses	Current-session shared records appear
8	Calculate member balances	Net balances are displayed
9	Generate fairness report	Amounts to pay or receive are displayed
10	Enter an invalid menu option	Invalid-choice message appears
11	Restart the program and view old records	Old records are not automatically loaded into memory

12.1 Result Analysis

The application is designed to perform all seven menu operations.

The personal expense functions demonstrate the recording and summation of individual expenses. The shared expense functions demonstrate group expense recording and equal division of costs.

The fairness calculation processes each shared expense, calculates each participant's share, credits the payer with the amount paid, and displays the resulting net balance.

The current implementation also saves new records to text files. However, the saved records are not automatically loaded when the program starts again.

Before submission, run each test case and confirm the actual results. Insert screenshots of successful program execution as supporting evidence.

13. LIMITATIONS

Although the application fulfils its basic objectives, it has several limitations.

1. **Console Interface:** The application uses a text-based menu instead of a graphical interface.
2. **No Automatic Data Loading:** Records are saved in text files but are not loaded into memory when the program restarts.
3. **Session-Based Summary:** Personal spending totals include only expenses entered during the current session.
4. **Session-Based Shared Balances:** The fairness report includes only shared expenses entered during the current session.
5. **Equal Sharing Only:** Every expense is divided equally among the participating members.
6. **No Actual Payment Processing:** The program calculates balances but does not transfer money or record completed payments.
7. **Basic File Format:** The text-file format uses delimiters and may not reliably handle descriptions containing those delimiters.
8. **No Separate Group Management:** The current version does not support multiple independently managed groups.

14. FUTURE SCOPE

The application can be improved in several ways.

14.1 Persistent Data Loading

The program can be updated to read existing records from text files at startup so that users can view earlier expenses and calculate cumulative totals.

14.2 Monthly Spending Reports

Date fields can be added to expenses to generate daily, weekly, and monthly spending reports.

14.3 Category-Wise Analysis

The application can calculate how much money is spent on categories such as food, travel, education, and shopping.

14.4 Unequal Expense Sharing

Future versions could allow different members to contribute different shares rather than dividing every expense equally.

14.5 Settlement Suggestions

The application could generate a list of suggested transfers to settle outstanding balances while minimising the number of transactions.

14.6 Graphical User Interface

A graphical interface could make the application easier to use and improve the presentation of expense reports.

14.7 Data Export

The application could support exporting expense records and reports to CSV or PDF files.

14.8 Database Integration

A database could replace plain text files, providing more structured storage and easier searching and retrieval of historical records.

15. CONCLUSION

The Personal Expense Tracker with Shared Expense Fairness Detection is a C++ mini project designed to address common problems in personal spending and group expense management.

The application allows users to add personal expenses, view expense records, calculate personal spending totals, add shared expenses, view shared expenses, calculate member balances, and generate a fairness report.

The main feature is the Shared Expense Fairness Detector. It calculates the net balance of each member by dividing shared expenses equally among participating members and accounting for the amount paid by each person. This makes it easier to understand who should pay money and who should receive money.

The project demonstrates practical applications of classes, objects, constructors, encapsulation, vectors, maps, loops, conditional statements, and file handling.

Although the current version has limitations, such as not automatically reloading saved records, it provides a useful foundation for future improvements.

Overall, the project shows how a simple C++ application can apply programming concepts to a real-world financial problem while improving the clarity of shared expense calculations.

16. REFERENCES

1. Stroustrup, Bjarne. *Programming: Principles and Practice Using C++*. Addison-Wesley.
2. Stroustrup, Bjarne. *The C++ Programming Language*. Addison-Wesley.
3. cppreference.com. *C++ Language and Standard Library Reference*. https://en.cppreference.com/w/
4. GCC Project. *GCC Documentation*. https://gcc.gnu.org/
5. C++ Standard Library documentation covering vectors, maps, file streams, strings, and input/output operations.
6. Project source code: main.cpp, Personal Expense Tracker with Shared Expense Fairness Detection.